

Data Structure and Object-Oriented Programming

Final Project Report

HETERO - Human Efficiency Through Extensive Realtime Outreach

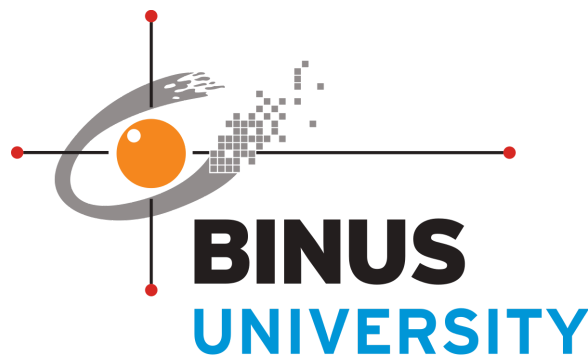
Task Management Tool

By:

Dyka Adlero Clifzier Holy Covenant (2902719013)

Ryan Richard Kalona (2902718774)

Simran Ramchandani (2902710973)



Class L2BC

Computer Science Program

School of Computing and Creative Arts

Bina Nusantara International University

Jakarta

2026

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
A. BACKGROUND.....	3
B. PROBLEM.....	3
C. SOLUTION.....	4
1. Sequential Task Management via ArrayList.....	4
2. Dynamic Task Queue via LinkedList.....	4
3. Instant Task Lookup via HashMap.....	5
D. RESULTS.....	5
1. Add/Insert Tasks (Bulk).....	5
2. List All task (FindAll).....	6
3. Updating task details (update).....	6
4. Find Task by ID (findByld).....	7
5. Filter by Completion Status.....	7
6. Filter by task due today.....	8
7. Filter by Category.....	8
8. Filter by Priority.....	9
9. Delete Task (delete).....	9
10. Graphs.....	10
11. Analysis.....	13
11.1. Adding / Insertion Task.....	13
11.2. List All Task.....	14
11.3 Updating Task Details.....	16
11.4 Finding Task by ID.....	17
11.5 Filter by Completion Status.....	17
11.6 Filter by Task Due Today.....	18
11.7 Filter by Category.....	19
11.8 Filter by Priority.....	19
11.9 Delete Task.....	20
11.10 Summary.....	21
E. IMPLEMENTATION.....	22
1. UML Class Diagram.....	22
2. Evidence of Working Program.....	23
REFERENCES.....	28
APPENDIX.....	28

A. BACKGROUND

The practice of task tracking dates back to the 18th century, when Benjamin Franklin popularized the use of to-do lists as part of his disciplined daily routine. Franklin's lists of virtues and responsibilities exemplified how structured planning could balance the demands of a statesman and inventor [1][4]. Originally, these lists were simple handwritten notes, often marked with checkboxes beside each task. Over time, the simplicity of paper lists gave way to increasingly sophisticated systems, culminating in modern digital task management tools (TMTs) [2].

Research shows that individuals lose an average of 3.5 hours per day without effective task management systems [3], underscoring the importance of digital solutions. Unlike traditional paper lists, digital TMTs are not limited by physical constraints such as paper size. Their capacity is virtually limitless, expanding according to the capabilities of the user's device. Moreover, when tasks are scattered across multiple physical notes, maintaining an ergonomic and efficient workflow becomes challenging, particularly in collaborative environments [6]. Digital TMTs consolidate all tasks into a single platform, enhancing organization, accessibility, and completion rates [3][5].

This project, **HETERO**, builds on that legacy by leveraging efficient data structures to optimize task operations. By comparing implementations such as ArrayList, LinkedList, and HashMap through Big-O analysis, HETERO aims to replicate and enhance conventional to-do lists with measurable performance gains. In doing so, it aligns with both historical principles of disciplined task tracking and contemporary research on productivity frameworks [2][5].

B. PROBLEM

As outlined in the background, managing multiple tasks efficiently remains a persistent challenge for individuals and organizations. Traditional approaches, such as handwritten to-do lists or basic digital tools, often lack the advanced capabilities required to support complex workflows, dynamic prioritization, and real-time collaboration. Modern users increasingly expect systems that not only record tasks but also enable seamless updates, notifications, and integration across devices.

However, current task management tools frequently encounter scalability and performance limitations when handling large volumes of tasks or multiple users simultaneously. Inefficient data structures can result in slow response times, excessive memory consumption, and difficulty maintaining an organized workflow. These shortcomings contribute to missed deadlines, reduced productivity, and heightened user frustration.

The conventional task management tools usually rely on a singular dynamic array (standard Python list or JavaScript Array) to manage tasks in a contiguous block of memory

[7]. Even though these basic structures can decently handle end-of-list actions such as `.append()` or `.push()`, a significant bottleneck [8] is apparent when more advanced mechanics such as marking a task as accomplished, deleting a task, or filtering tasks based on their categories. Furthermore, when a user filters, deletes, or marks a task complete, conventional array-based apps are compelled to invoke a memory-shift operation, urging the CPU to move every task one slot over to close or open a gap in memory. This operation is an $O(n)$ and is a main issue that slows down the structural process, drains the user's battery, and delays task execution.

To address these issues, this project will implement and evaluate several data structures optimized for task management operations. The objective is to identify the most effective structure that balances runtime efficiency and memory usage while supporting essential functionalities, including task creation, deletion, updating, prioritization, and progress tracking.

Specifically, the project will analyze the performance of data structures such as Linked Lists, Hash Maps, and Array Lists. Their time and space complexity will be assessed for key operations:

- Add/Insert Tasks (Bulk)
- List All Tasks (FindAll)
- Update Task Details
- Find Task by ID (findById)
- Filter by Completion Status
- Filter by Tasks Due Today
- Filter by Category
- Filter by Priority
- Delete Task (Delete)

By measuring average runtime and memory consumption for these operations, the project aims to determine the optimal data structure for building a scalable, responsive, and user-friendly task management system. This evaluation will provide a clear comparison of trade-offs, ensuring that the chosen structure supports both efficiency and usability in practical task management scenarios.

C. SOLUTION

Crafting a Task Management Tool that is capable of more than a mere placeholder for mental tasks but sophisticated features like editing tasks on the go, adding a deadline with `DateTime`, and even different filters for different usages demands data structures beyond arrays and lists. Knowing that, we decided to implement specialized types of data structures, that will be further elaborated:

1. Sequential Task Management via ArrayList

When the app is set to the `ArrayList` mode, the tasks are stored sequentially in a contiguous block of memory. This `ArrayList` engine performs exceptionally well in rendering UI components, such as the main dashboard, timeline, or the scrollable tasks. However, `ArrayList` possesses its limitations: when a critical-priority task is inserted into the middle of the list or deleting a task makes the system to shift all the subsequent elements in the memory. This results in $O(n)$ time complexity. This is especially true when the number of requests (tasks) grows [10].

2. Dynamic Task Queue via LinkedList

When the toggle is switched into a LinkedList engine, the tool becomes a sequential chain of independent nodes. Each task profile is individually encased inside node containing pointers that refer to the immediate preceding and succeeding task in the environment. However, although this mode shuffles task quickly, drop items, or add task with commendable speed, index-based random access is impossible, features like finding a specific task by ID, it is obliged to scan the chain sequentially from head to node, resulting in a $O(n)$ lookup time [9].

3. Instant Task Lookup via HashMap

Switching the mode into HashMap organizes the app into an extremely efficient collection of uniquely encoded key-value pairs. A special identifier, such as taskID or special task titles, acts as the key that directly maps the data needed to the corresponding task objects with full technical specification and deadline. This design is superior since it eliminates sequential search across all features. With the unique key through a hashing function, the HETERO TMT (task management tool) can spot the exact memory containing the task, easily securing a $O(1)$ for not only the basic CRUD operation (Create, Read, Update, Delete), but also the more advanced filtering features that displays tasks based on their order of importance, deadline, categories, etc. The major bottleneck is that it does not have a sequential order, thus, it requires a separate logic when the UI needs an alphabetical view of the task [11].

D. RESULTS

In pursuit of the superior data structure for our Task Management Tool, we tested the three data structures in terms of each time and memory of the HashMap-based structure, ArrayList-based structure and LinkedList-based structure. Each structures are then subjected to gradual stress-testing from 2,500, 5000, 7500, and eventually 10,000 across all of the listed operations: add/insert task in bulk, list all task, update task details, find task by ID, filter task by completion status, filter task by due today (via DateTime), and filter by category. Every feature is tested in all three data structures to distill the ultimate data structure that would be the best for our Task Management tool.

1. Add/Insert Tasks (Bulk)

Average time taken for x task (ms)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
Hashmap	3.7920	3.1060	7.4874	26.3868	10.19305
LinkedList	6.5292	10.2043	3.8664	5.3393	6.4848
ArrayList	4.8828	5.0384	2.8753	4.0927	4.2223

Average memory used for x task (kilo-bytes)					
Data Structure	2500	5000	7500	10,000	Average (for all average)

HashMap	813.84	1558.35	2336.94	2523.55	1808.17
LinkedList	813.84	1394.14	2048.00	2743.66	1749.91
ArrayList	813.94	1349.67	1991.52	2675.75	1707.72

2. List All task (FindAll)

Average time (ms)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
HashMap	0.3845	0.4567	0.6325	0.2130	0.421675
LinkedList	0.1106	0.4226	0.3202	0.6605	0.378475
ArrayList	0.1176	0.1630	0.3281	0.1858	0.198625

Average memory used (kilo-bytes)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
HashMap	69.99	51.33	61.12	92.92	68.84
LinkedList	69.99	51.91	61.40	112.40	73.925
ArrayList	69.99	54.77	63.82	111.26	74.96

3. Updating task details (update)

Average time (ms)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
HashMap	0.0544	0.0688	0.0612	0.0464	0.0577
LinkedList	0.3478	0.3039	0.6275	0.5503	0.4574
ArrayList	0.1282	0.2196	0.3261	2.8822	0.8890

Average memory used (kilo-bytes)					
Data Structure	2500	5000	7500	10,000	Average (for all average)

HashMap	0.1044	31.80	31.82	9.96	18.4211
LinkedList	60.24	32.39	32.12	34.27	32.505
ArrayList	0	0	0	0	0

4. Find Task by ID (findById)

Average time (ms)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
HashMap	0.0106	0.0061	0.0053	0.0106	0.0081
LinkedList	0.3323	0.2825	0.3740	0.3779	0.3417
ArrayList	0.1680	1.3184	1.9501	2.3442	1.4452

Average memory used (kilo-bytes)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
HashMap	60.23	31.80	31.82	6.75	32.65
LinkedList	60.24	32.39	32.12	34.27	39.755
ArrayList	60.24	35.26	34.54	33.13	40.7925

5. Filter by Completion Status

Average time (ms)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
HashMap	0.3099	0.4687	0.5497	0.3298	0.4145
LinkedList	0.2178	0.3974	0.4127	0.3218	0.3374
ArrayList	0.1672	0.3617	0.5008	2.8451	0.9687

Average memory used (kilo-bytes)					
Data Structure	2500	5000	7500	10,000	Average (for all average)

HashMap	60.25	60.25	31.84	34.20	46.635
LinkedList	60.23	60.24	32.12	34.27	46.715
ArrayList	60.24	60.24	34.54	43.99	49.7525

6. Filter by task due today

Average time (ms)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
HashMap	0.2810	0.3588	0.3803	0.3858	0.3515
LinkedList	0.1964	0.3459	1.3920	0.3210	0.5638
ArrayList	0.1927	0.3854	0.5077	1.4752	0.6402

Average memory used (kilo-bytes)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
HashMap	60.27	31.82	42.72	36.92	42.9325
LinkedList	60.23	32.39	43.00	34.30	42.48
ArrayList	60.23	35.26	34.56	44.02	43.5175

7. Filter by Category

Average time (ms)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
HashMap	0.4436	0.4517	0.3151	0.3488	0.3898
LinkedList	0.1126	0.3472	5.9615	0.2715	1.6732
ArrayList	0.1730	0.3537	0.6609	0.5235	0.4278

Average memory used (kilo-bytes)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
HashMap	60.25	42.68	58.98	74.71	59.155
LinkedList	60.24	43.25	59.26	74.97	59.43
ArrayList	60.24	35.26	50.82	84.69	57.7525

8. Filter by Priority

Average time (ms)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
HashMap	0.2261	0.3374	0.3224	0.3058	0.2979
LinkedList	0.0975	0.2766	1.0253	9.6417	2.7603
ArrayList	0.1484	0.2852	0.3885	1.4180	0.5600

Average memory used (kilo-bytes)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
HashMap	60.25	31.82	42.70	50.30	46.2675
LinkedList	60.24	32.39	42.98	50.55	46.54
ArrayList	60.24	35.26	34.54	60.27	47.5775

9. Delete Task (delete)

Average time (ms)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
HashMap	0.0066	0.0080	0.0063	0.0165	0.0094

LinkedList	0.1681	0.4818	2.2049	2.3121	1.2917
ArrayList	2.2129	0.4285	0.6789	0.9718	1.0730

Average memory used (kilo-bytes)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
HashMap	60.23	31.80	31.82	2.58	31.6075
LinkedList	60.23	32.38	32.10	74.97	49.92
ArrayList	60.23	35.24	34.52	33.12	40.7775

10. Graphs

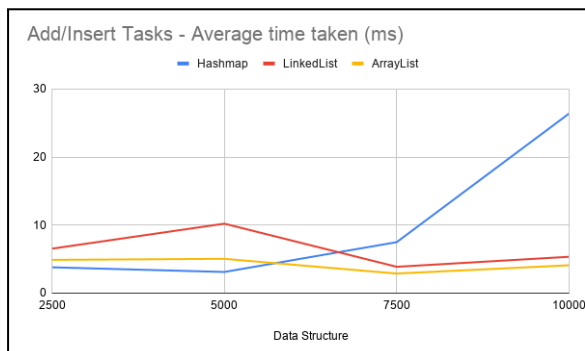


Figure 1. Average Time Taken Graph
(Adding Tasks)

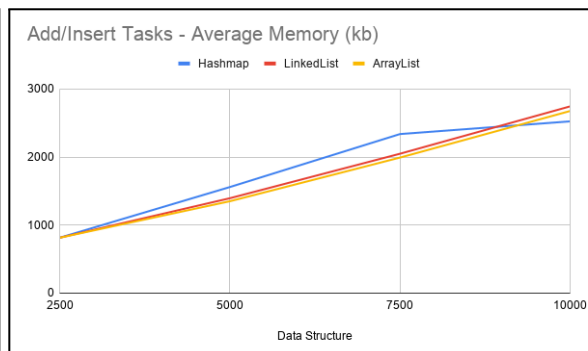


Figure 2. Average Memory Used Graph
(Adding Task)

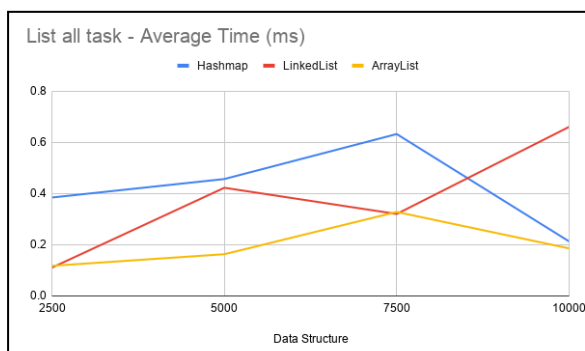


Figure 3. Average Time Taken Graph
(List all task)

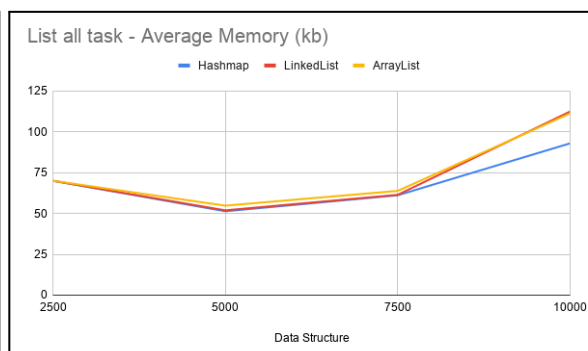


Figure 4. Average Memory Used Graph
(List all task)

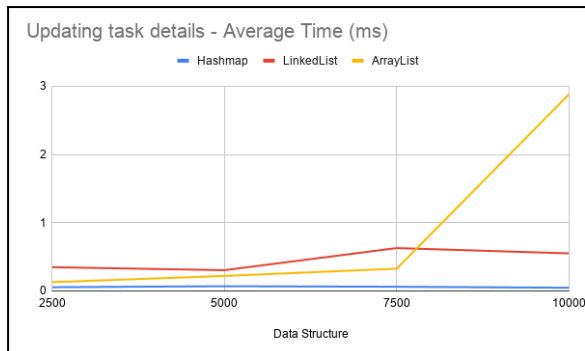


Figure 5. Average Time Taken Graph
(Update task)

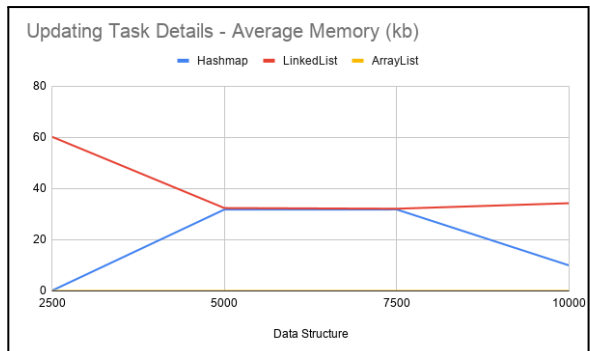


Figure 6. Average Memory Used Graph
(Update task)

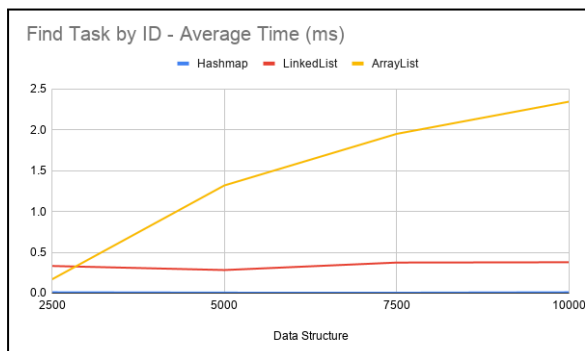


Figure 7. Average Time Taken Graph
(Finding Task by ID)

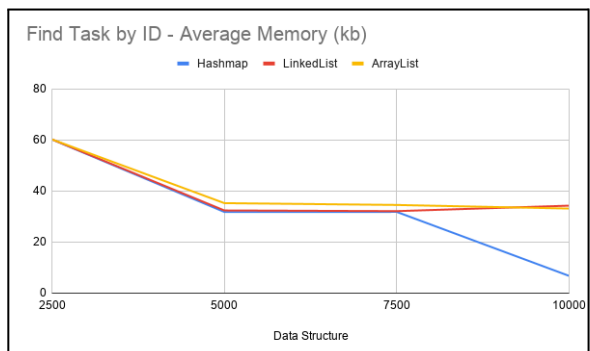


Figure 8. Average Memory Used Graph
(Finding Task by ID)

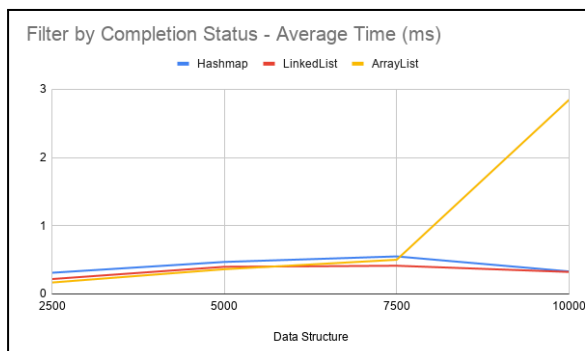


Figure 9. Average Time Taken Graph
(Filter by Completion Status)

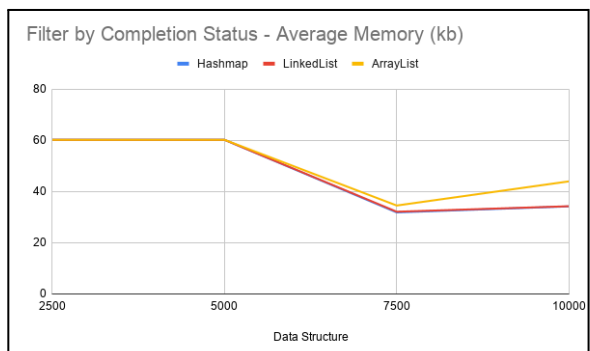


Figure 10. Average Memory Used Graph
(Filter by Completion Status)

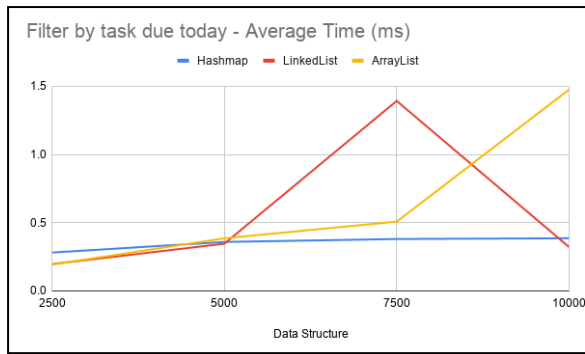


Figure 11. Average Time Taken Graph
(Filter by Task Due Today)

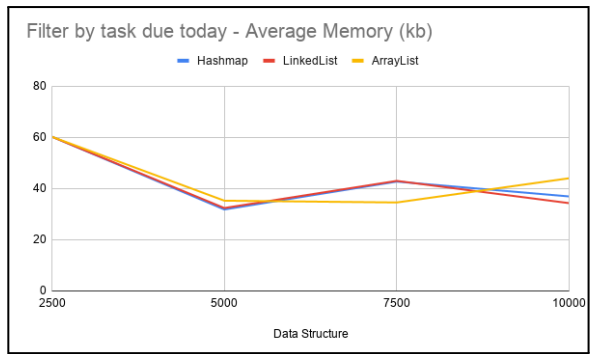


Figure 12. Average Memory Used Graph
(Filter by Task Due Today)

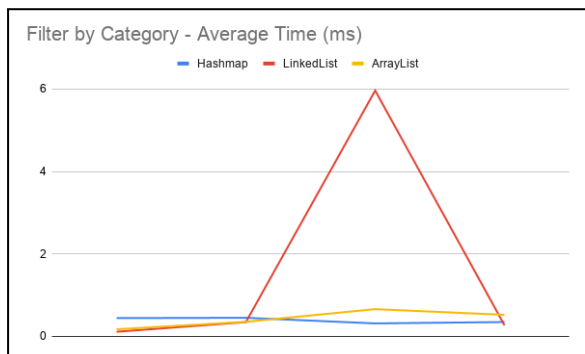


Figure 13. Average Time Taken Graph
(Filter by Category)

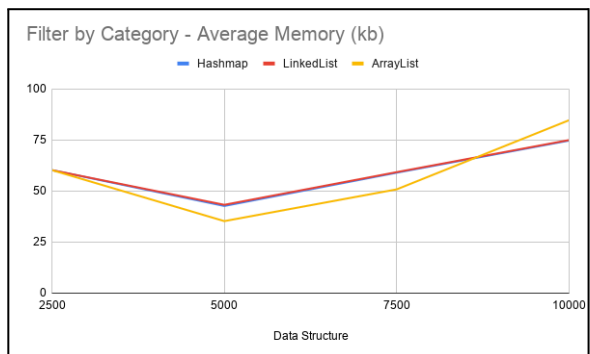


Figure 14. Average Memory Used Graph
(Filter by Category)

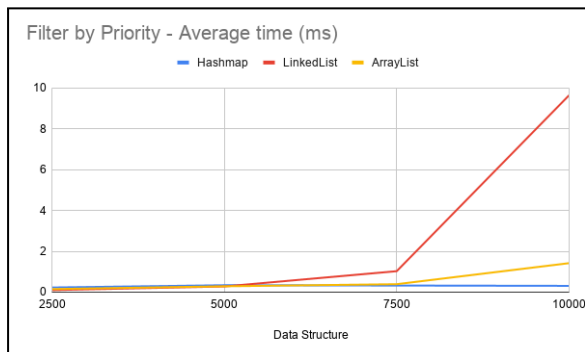


Figure 15. Average Time Taken Graph
(Filter by Priority)

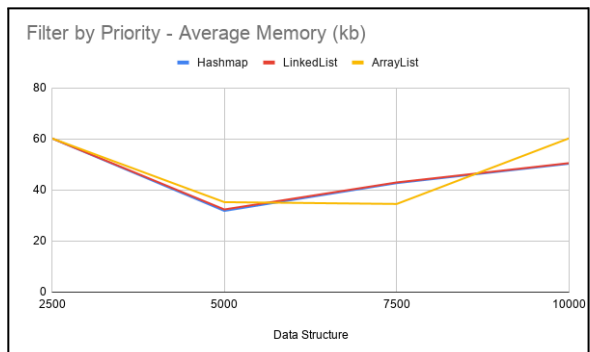


Figure 16. Average Memory Used Graph
(Filter by Priority)

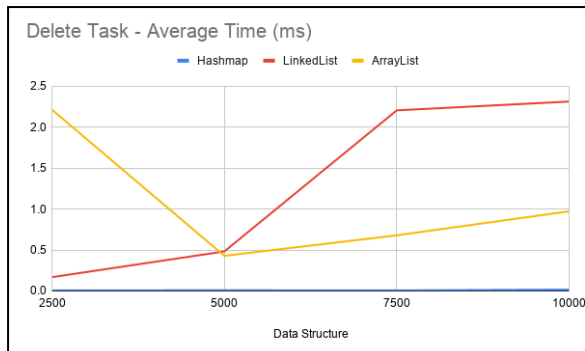


Figure 17. Average Time Taken Graph
(Delete Task)

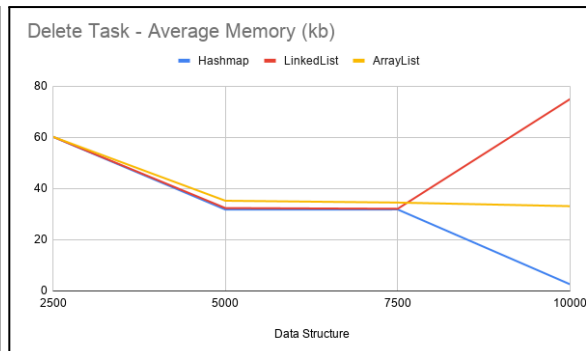


Figure 18. Average Memory Used Graph
(Delete Task)

11. Analysis

The data above depicts each detailed time and space performance under 2500, 5000, 7500, and 10,000 requests. The following section discusses the why behind every rise and drop of the performance. The initial part of the description of each section will hover about the numerical findings in the stress test, while the latter will explore its theoretical validity. Through the course of this analysis, there will be a section (usually a paragraph or more) dedicated to comprehensively break down any quirks in the numbers found in the tables above.

11.1. Adding / Insertion Task

Time - figure 1

As shown above, the HashMap is rapid when subject to small requests (below 7500), maintaining the amortized $O(1)$ operation with an average runtime of 10.19305 millisecond [9][12]. The data displays a spike as the number of requests increase because the HashMap triggers a resize to keep the $O(1)$ performance. This resize operation is $O(n)$ [14]. On the other hand, the seemingly random fluctuation of the LinkedList [12] (6.5, 10.2, 3.8, 5.3 millisecond) is mostly because its nodes are scattered (not contiguous) in memory [7][8]. Therefore, the CPU cannot predict memory location as easily as in an array. When comparing it to the rapid average time of the ArrayList of 4.2223 milliseconds, we can conclude that for this specific insertion task, ArrayList is superior [10]. The improvement in 7500 insertions (2.8753 milliseconds) was due to external factors such as garbage collection cycles or CPU cache saturation rather than the behaviour of the data structures itself [15][16].

Choosing ArrayList as the fastest implementation for this insertion task is justifiable because it appends the elements at the end, making it incredibly efficient [8]. LinkedList, in contrast, must allocate a new node object in the heap each time an insertion was made, loading the memory and cache [7]. Meanwhile, for every input HashMaps need to handle potential bucket collisions and calculate hash codes [14]. In summary, ArrayList is the fastest due to its pure structural simplicity given you don't insert at the beginning, forcing it to shift elements [10].

The anomalies found in the numbers will be discussed:

1. Runtime is faster as tests grow
 - The List is faster at 10,000 than at 2,500 items
 - This is driven by while (true) loop in PerformanceTest.java:

```

while (true) {
    System.out.println(x: "=====");
    System.out.println(x: "      COMPLEXITY ANALYSER & BENCHMARK TOOL      ");
    System.out.println(x: "=====");
    System.out.println(x: "Select a Data Structure to evaluate:");
    System.out.println(x: "1. ArrayList (ArrayListTaskRepo)");
    System.out.println(x: "2. LinkedList (LinkedListTaskRepo)");
    System.out.println(x: "3. HashMap (HashMapTaskRepo)");
    System.out.println(x: "4. Run All Structures (Side-by-Side Comparison)");
    System.out.println(x: "5. Exit Program");
    System.out.print(s: "Enter your choice (1-5): ");
}

```

Note: only a snippet is displayed since it will take up the page count

- When the test is run sequentially in the same session, JVM detects that the .add() method are executed over and over, JIT Compiler [13] compiles these hotspots into optimised native machine code, making them (10,000 insertions) to look faster than the 2,500-item.
2. The HashMap's significant increase in 10,000
 - The HashMaps' runtime significantly increases from 7.4874 milliseconds into 26.3868 milliseconds
 - This is triggered in PerformanceTest.java when calling repo.add(task) which wraps HashMap.put() in HashMapTaskRepo.java
 - Since HashMap here is initialized with the default capacity of 16 buckets, the HashMap performs array relocation and rehashing [14].

Memory usage - figure 2

A benchmark evaluation during the task insertion shows that the ArrayList is the most space-efficient data structure with an average of 1707.72 KB [10]. This consistently outperforms both LinkedList (1749.91 KB) and HashMap (1808.17) in the stress test [9].

The anomalies in the memory are as the following:

1. 0KB Memory on ArrayList

- Due to overwriting in ArrayListTaskRepo.java
- When updating with taskStore.set(currentIndex, updatedTask), the code overwrites an existing pointer without instantiating any new wrapper object [8], thus making it exactly 0 KB.

11.2. List All Task

Time - figure 3

The reason why LinkedList is the fastest on average as well as on 2,500 and 7,500 is because the recently inserted data is still available in the CPU cache, triggering a quicker access [10]. LinkedList benefits from the cache hits while avoiding the array-reining in ArrayList because the findAll() runs immediately after insertion [8].

In the LinkedListTaskRepo implementation in findAll(),
 List<Task> result = List.copyOf(taskStore);

At a smaller scale, traversing a LinkedList is more rapid than that of an array because the amount of data is still small and the memory access is still efficient [7]. The CPU also does not experience major delays.

Interestingly, an outlier was found in the HashMap when the operation hits 10,000. As seen in the table below:

Average time (ms)					
Data Structure	2500	5000	7500	10,000	Average (for all average)
Hashmap	0.3845	0.4567	0.6325	0.2130	0.421675
LinkedList	0.1106	0.4226	0.3202	0.6605	0.378475
ArrayList	0.1176	0.1630	0.3281	0.1858	0.198625

The runtime drops from 0.6325 (7500 operations) to 0.2130 milliseconds in 10,000 operations. There are several answers to justify this anomaly. Below is the detailed breakdown:

a. The Asynchronous Garbage Collector

In PerformanceTest.java, all features forces a garbage collector suggestion right before opening the stopwatch:

```
System.gc();
memBefore = runtime.totalMemory() - runtime.freeMemory();
start = System.nanoTime();
repo.findAll();
long listTime = System.nanoTime() - start;
memAfter = runtime.totalMemory() - runtime.freeMemory();
long listSpace = Math.max(0, memAfter - memBefore);
```

[System.gc\(\)](#) continues running while the Garbage Collector works, since it is a non-blocking request [15]. At lower task count (2,500-7,500), the heap pressure is minimal. The JVM thus executes a quick background sweep [16].

That is why when `start = System.nanoTime()` starts, the background garbage collector threads are running as well, interfering with the CPU cycle, polluting the L1/L2 hardware cache, and changes the execution time significantly [15].

b. Memory Scale and Stop-the-World Sweeps

Whenever a certain dataset reaches 10,000 requests, the setup process produces more temporary data and memory usage significantly soars.[15] At this moment, JVM considers [System.gc\(\)](#) more important and performs a full Stop-the-World garbage collection [14][15], pausing all application threads temporarily. Since the garbage collection finishes before the benchmarking was conducted, the system starts in a sterile and non-volatile state [16].

Therefore, `repo.findAll()` runs with less background activity and makes it faster than expected although the dataset is massive.

c. HashMap Collection Isolation with CPU Cache effect

The `findAll()` method in `HashMapTaskRepo` creates a new list from all values stored in the `HashMap`:

```
@Override
public Optional<Task> findById(int id) {
    long startTime = System.nanoTime();
    Optional<Task> result = Optional.ofNullable(taskStore.get(id));
    printBenchmark(operationLabel: "FindById", System.nanoTime() - startTime);
    return result;
}
```

Because the `List.copyOf(taskStore.values())` operation must iterate through all tasks in `HashMap`, it is an $O(n)$ operation [9][12].

At 7,500 tasks, the background garbage collector activity competes for CPU resource and cache space, causing the processor to slow down. This increases the execution time to 0.2130 ms.

At 10,000 tasks, garbage collection may have been completed before the benchmark started, leaving the system with a smoother, fewer-interruption state, resulting in the low execution time of 0.2130 ms [16].

The reasons above explain the anomaly of the drop in runtime shown above.

Memory usage - figure 4

- ArrayList (Optimal: 1707.72 KB Average)
- LinkedList (Medium: 1749.91 KB Average)
- HashMap (Ineffective: 1808.17 KB Average)

Memory usage in ArrayList:

- The contiguous memory allocation is the main factor of its efficiency
- ArrayList stores reference sequentially when inserting a task without requiring wrapperobject [10]
- Minimal overhead even when inserted with 10,000 tasks (2675.75 KB)

Memory usage in LinkedList:

- Since it is node-based, it has a high memory footprint.
- Every iteration requires creating Node object that holds task data with backward pointer
- Accumulate significantly at 2743.66 KB when subject o 10,000 tests

Memory usage in HashMap

- Requires wrapping data to internal Entry object with its hash value
- Pre-allocates internal bucket array
- Leaves a portion of its capacity empty because of its load factor

11.3 Updating Task Details

Time - figure 5

HashMap is superior (0.0577 millisecond average) in this operation due to the way it is implemented [9]. In the code below:

```
@Override
public void update(Task task) {
    long startTime = System.nanoTime();
    taskStore.put(task.getId(), task);
    printBenchmark(operationLabel: "Update", System.nanoTime() - startTime);

    if (isDbEnabled) {
        databaseManager.update(task);
    }
}
```

- task.getId() fetches a specific task by its unique identifier and replace/update it in the collection
- Uses hashing function to locate the exact bucket where the key belongs: O(1) constant complexity[12][14], allowing it to insert or overwrite the value in near instant with no regards to the size of the dataset [12].

LinkedList and ArrayList neither support this key-value mapping behaviour [9]. Therefore, they should mimic this behaviour by searching the collection sequentially, element-by-element to find the task matching task.getId() [8][10].

In summary, HashMap finishes the operation quickly since it can jump directly to the correct item [14], while ArrayList is slowed down by the searching process at (2.8822 millisecond at 10,000 tasks) which is approximately 50 times slower [7].

Some peculiar figures are summarized below:

1. ArrayList 10,000 items spike

- The performance of ArrayList slows down to an extreme (2.88222 millisecond), which is 50 times slower than its 2,500 tasks.
- We used the classic for-loop linear scan to find the task index before we update it [7][10]. At 10,000 items, the sequential block encounters a cache-missing overhead [7].

```
for (int currentIndex = 0; currentIndex < taskStore.size(); currentIndex++) {
    if (taskStore.get(currentIndex).getId() == updatedTask.getId()) {
        taskStore.set(currentIndex, updatedTask);
        break;
    }
}
```

The ArrayList for-loop that eventually slows down in 10,000 tasks

Memory usage - figure 6

ArrayList displays a 0 in memory because of the internal array being utilized or garbage collection variation (which is quite recurrent in this research). HashMap maintains a stable and stable record. Regardless, ArrayList is still the most efficient due to its zero value in memory.

Furthermore, HashMap took 18.4211 KB of memory on average. This figure is significantly lower than that of the LinkedList (32.505). Since LinkedList wraps every task into a special node object with overhead for next and prev pointers, it consumes more memory overall.

11.4 Finding Task by ID

Time - figure 7

When discussing finding a certain task by its ID, it is apparent that HashMap is operating in its homebase. With an extremely swift performance of just (0.0081 millisecond) on average, there are several theoretical validities that can be proven through the behaviours of HashMaps:

- In HashMap, the time stays the same (between 0.0106 milliseconds and 0.0053 milliseconds) disregarding the fact that they are subject through 2,500 or 10,000 items. This result is valid because HashMaps utilizes a key-to-bucket mechanism [12][14]. This allows it to leap directly to the requested ID instantly, an O(1) operation [12].
- ArrayList stumbles when the number of the requests grows from 2,500 to 10,000. When it is only 2,500 tasks, the time is around 0.1680 milliseconds, but when it is stress-tested with 10,000 tasks, the time balloons to 2.3442 milliseconds. The reason behind that is because ArrayList does not have “keys” as a concept. Therefore it must examine every single slot one-by-one from the beginning until it finds the matching ID [7][10].
- LinkedList is the slowest and the most unstable (0.3417 millisecond average) since it has to search sequentially like ArrayList, making it very ineffective for ID Lookups [8][10].

Memory usage - figure 8

For a findById operation, HashMap is still undisputed to both ArrayList and LinkedList. HashMap achieves lookups that are around 42 times faster than LinkedList and 178 times faster than ArrayList. As a conclusion, this findById is a classic textbook example that demonstrates the hashing function that locates items by its unique ID [14].

11.5 Filter by Completion Status

Time - figure 9

When testing the filtering by completion status, LinkedList is most effective in filtering tasks, marking it based on its completion status. With a 0.3374 milliseconds average, it slightly outperforms HashMap with 0.4145 milliseconds and ArrayList with 0.9687 milliseconds.

This result is due to the linear scan with boolean flag ($O(n)$ complexity where each single element in the collection has to be inspected [7][10]. Regarding ArrayList and HashMap, the time allocated remains linear as the dataset expands, with both averaging 0.32 milliseconds at 10,000 tasks.

HashMap is slower since it should utilise `taskStore.values().stream()`, pulling entries into a stream. In relation to this, the LinkedList is slower [8].

Some note-worthy findings are summarized below:

1. ArrayList collapses at 10,000 items
 - Up to 7,500 tasks, ArrayList is still efficient (0.5008 milliseconds)
 - When it hits 10,000 tasks, it jumps to 2.8451 milliseconds
 - When looking at `.collect(Collectors.toList())`, the stream passes items into a dynamic backing array. This is done when a stream builds a list from an unknown target size. This forces the internal array to expand, erasing the data structure's cache locality advantage.

Memory usage - figure 10

We can see that the HashMap was the most effective data structure for memory in this filtering operation. HashMap uses an average of 46.635 KB, followed by LinkedList (46.715 KB) and ArrayList (49.7525 KB).

This can be justified by examining the allocation behaviour of Java streams and memory management during sequential evaluation. A peculiar anomaly was found where all three data structures take more than 60 KB of memory at the 2,500 and 5,000 tasks, but their footprints drop by nearly half (around 31-34 KB) when scaling to 7,500 tasks. This happens because the benchmark implements `System.gc()`. This calls immediately before tracking the heap space delta. In a smaller scale, the internal JVM optimizes the execution path, allowing the system to reuse memory or clean up garbage frames with extreme speed before the final heap calculation [8].

The ArrayList eats up the most memory, especially in the 10,000 threshold (43.99 KB). This is directly correlated to its degradation in runtime. The dynamic array expansion in the collector pipeline creates overhead on the heap. Compare this to the cleaner arrays utilized when the streaming values of HashMap or traversing LinkedList. In conclusion HashMap is superior since it provides the most balanced performance across operations [9].

11.6 Filter by Task Due Today

Time - figure 11

From the table and graph result, again HashMap was proven as the fastest data structure for filtering "task today". With an average time of 0.3515 milliseconds, hashmap excels beyond LinkedList (0.5638 milliseconds) and ArrayList (0.6402 milliseconds).

This behaviour can be explained by analyzing the structural mechanisms of accessing data sequentially for date evaluation. Since dates are examined across every record, all structures should perform a full linear traversal to check each task, which is an $O(n)$ operation[7][10]. The HashMap flexes an exceptional consistency in time, making it narrow and balanced (around 0.35 to 0.38 milliseconds both for small tasks up to 10,000 tasks) [9]. This flat consistency is again made possible by the JIT compiler optimization. The compiler maps the structural value of the map itself to loop directly to the native CPU code.

The LinkedList exhibits a peculiar performance at 7,500 tasks (it spikes at 1.3920 milliseconds, but mellowed out in 10,00 tasks with 0.3210 milliseconds). This classic spike can be blamed upon the garbage collection (GC) interference when the runtime was measured. Because `PerformanceTest.java` depends on active system timing blocks, if the JVM triggers a background GC sweep in the middle of traversing, the entire thread pauses briefly,

creating a false performance drop [15]. As a conclusion, the HashMap is still the most reliable structure for filtering due to its lack of loop traversal friction [9].

Memory usage - figure 12

It is apparent that from the tables above, LinkedList was the most memory-efficient for this filtering with an average memory consumption of 42.48 KB. HashMap ranks second at 42.9325 KB followed by ArrayList at 43.5175 KB.

This can be explained by looking at how date filter streams allocate matching collection outputs. Similar to a completion filter chart, a clear memory nosedive is visible as the data scales from 5,000 tasks to 7,500 tasks. At 2,500 requests, all structures experienced a uniform baseline of around 60.2 KB. When Java Streams (.stream().filter()) is used, Java is compelled to craft a temporary pipeline framework in memory to pass the data through. On top of that, LocalDate.now() creates a new date object just to run the check. Because of that, the memory used by the temporary framework is much heavier than the data structures themselves, thus making them start at the 60 KB baseline[15].

11.7 Filter by Category

Time - figure 13

For this operation, HashMap still wins with the runtime averaging in 0.3898 milliseconds [9]. The ArrayList follows with 0.4278 milliseconds, and lastly LinkedList with 1.6732 milliseconds.

HashMap and ArrayList are stable because of how the elements are stored in memory [9]. The ArrayList packs elements into sequential slots [8]. HashMaps have a clear traversal path [14].

A quirky performance was noticeable when the LinkedList was tested on 7,500 tasks. The runtime spikes to 5.9615 milliseconds before calming down to 0.2715 milliseconds at 10,000 tasks. This massive increase is a real example of the runtime GC interference[15][16]. Knowing this, the HashMap proves to be more reliable and predictable.

Memory usage - figure 14

ArrayList is the most memory-efficient overall for the filtering option with an average of 57.7525 KB, followed by HashMap at 59.155 KB and LinkedList at 59.43 KB. While ArrayList wins on overall average, HashMap is better at handling large streams with a more trustworthy balance[12].

11.8 Filter by Priority

Time - figure 15

We can clearly see from the table result that HashMap was the fastest data structures when it comes to filtering tasks according to their priority level (Minimal, Low, Medium, High, Critical), with an average of 0.2979 milliseconds [9]. The ArrayList performs with the average of 0.5600 and LinkedList came last with 2.7603 milliseconds average [10].

In order to filter tasks by a certain degree of priority, the code must perform a full linear traversal across every single record since matching tasks are scattered throughout the entire dataset. The HashMap handles this efficiently, maintaining a flat execution timeline between 0.22 and 0.33 milliseconds. Such display of consistency is achievable by the JVM's JIT Compile, which eyed the benchmark looping through the collection recurrently and optimizes the bytecode immediately to the machine code [13].

Some unusual figures, however, are also present. The LinkedList performance started off strong at the initial 2,500 task requests (it was dashing at 0.0975 milliseconds), yet after 10,000 tasks, the runtime skyrockets exponentially into 9.6417 milliseconds. This is an example of Cache Locality [7][8].

An ArrayList tightly packs its reference pointer in a continuous block of memory, allowing the CPU to retrieve large groups of elements in one grasp with its high-speed cache [8]. A LinkedList, however, diversifies its individual node objects arbitrarily across differing memory locations on the Heap [7][10]. When the filter loop attempts to tackle 10,000 sequentially, the CPU must wait constantly to look up each memory address pointer of each node. This creates endless Cache Misses that makes the process slower, leaving the HashMap as the most technically sound and viable option [9].

The behaviour is determined by how the traversal is written in the repository class (this example is from ArrayListTaskRepo.java):

```
@Override
public List<Task> findByPriority(Priority priority) {
    long startTime = System.nanoTime();
    List<Task> result = taskStore.stream()
        .filter(task -> task.getPriority() == priority)
        .collect(Collectors.toList());
    printBenchmark(operationLabel: "FindByPriority", System.nanoTime() - startTime);
    return result;
}
```

Memory usage - figure 16

We can analyze that the HashMap comes out as the most memory-efficient structure when filtering based on priority with an average of 46.2675 KB. LinkedList follows at 46.54 KB and ArrayList is at 47.5775 KB.

The behaviour is explainable by viewing closely how temporary objects are placed and tracked inside the method boundaries. When closely looking, a prominent memory quirk was present in the initial 2,500-task setting where all three data structures are recorded to be at 60.2 KB baseline, exactly like the Filtering by task that is due today. Again, the same reason applies. When using Java Streams, Java initializes an underlying structure on the heap to process the collection. The initial weight of this framework clogs the memory, eventually overpowering the structures themselves [15][16].

11.9 Delete Task

Time - figure 17

When looking at the process of removing a certain task, HashMap once again wins with an amazing flat average of 0.0094 milliseconds [9]. Next is the ArrayList at 1.0730 milliseconds and LinkedList at 1.2917 milliseconds [10].

The graph shows that the LinkedList displayed an unusual delay when dealing with 7,500 tasks (execution spiked to 2.2049 milliseconds). This soar happens since deleting in a LinkedList requires pointer search across node objects around the Heap [7][8]. The ArrayList also displayed a starting delay of 2.2129 milliseconds at 2,500 tasks due to “cold code” execution before the JIT compiler activated [13]. The HashMap stayed stable and consistent across all sizes making it the most reliable in the deletion operation [12].

Memory usage - figure 18

The most memory-efficient is HashMap (31.6075 KB). ArrayList scores 40.7775 KB and LinkedList scores 49.92 KB. There are some divergences that occur in 10,000 tasks. First, the HashMap freezes to 2.58 KB. The LinkedList extremely spikes to 74.97 KB [15].

1. Zero memory usage in ArrayList

- The operation modifies the contiguous reference array in-place [8]
- No new container wrappers are allocated [10]
- The resulting memory remains negligible or zero

2. Memory Spike in LinkedList

- Deleting reference disconnects nodes and leaves unlinked element fragments floating in the heap
- Those fragments are registered as active

3. High Efficiency in HashMap

- Removing elements instantly updates and compares internal bucket entries, making the JVM able to scale down to 2.58 KB [12][14].

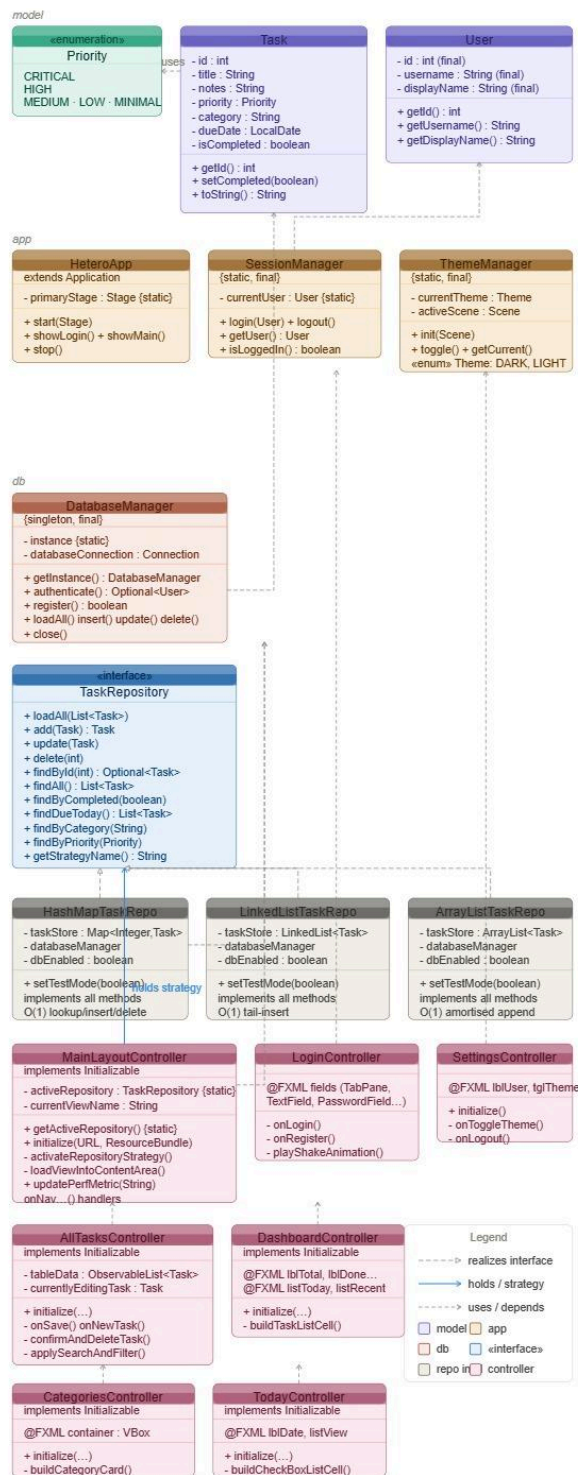
11.10 Summary

Across the entire benchmark analysis, HashMaps consistently comes out as the fastest and most reliable option for inserting, updating, deleting, and filtering specific tasks due to its key-bucket lookup speed which is $O(1)$ [9][12]. ArrayList excels at raw bulk because they append objects into contiguous memory slots, which solves the problem of node-creation and pointer-linking found in LinkedList or the array resizing bottleneck that slows down HashMaps [7][8]. Regarding memory, most filtering methods reveal near-identical benchmarks at smaller tests because the fixed weight of Java's streaming pipeline drowns the collection's native data structure [9][15], though ArrayList still maintains a steady, absolute zero KB change during the core update and delete modifications because it mutates pointer reference arrays wholly in-place[8][10].

E. IMPLEMENTATION

1. UML Class Diagram

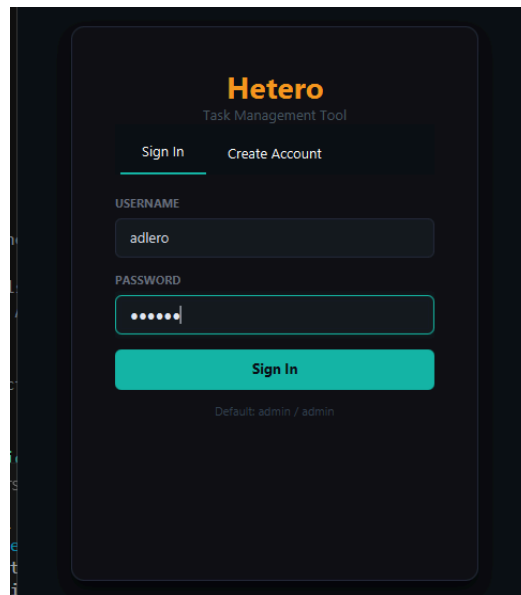
(Click [here](#) to view the image in high-resolution)



(The red square indicates private class, and the green circle indicates public class)

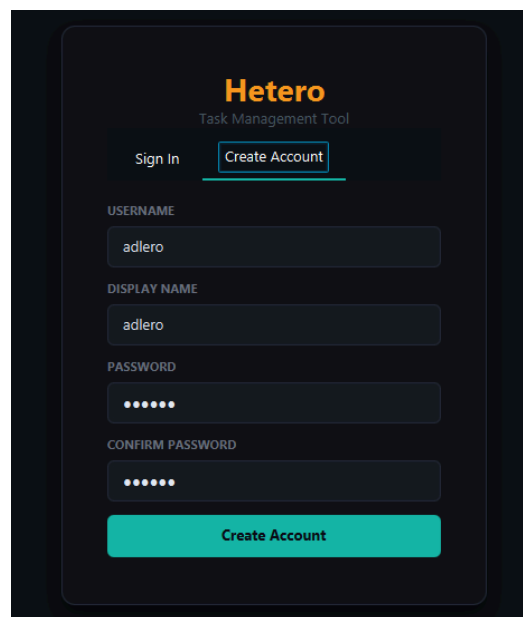
2. Evidence of Working Program

The following are our working program with and without the app:



The screenshot shows the login interface of the 'Hetero Task Management Tool'. At the top, the logo 'Hetero' is displayed in orange, with 'Task Management Tool' in smaller text below it. There are two tabs: 'Sign In' (which is selected and underlined in red) and 'Create Account'. Below the tabs, there are two input fields: 'USERNAME' containing the text 'adlero' and 'PASSWORD' containing six dots. A red 'Sign In' button is positioned below the password field. At the bottom, a small note states 'Default: admin / admin'.

Figure 19. Login Page



The screenshot shows the 'Create Account' interface of the 'Hetero Task Management Tool'. The logo 'Hetero' and subtitle 'Task Management Tool' are at the top. The 'Create Account' tab is selected and underlined in red, while the 'Sign In' tab is unselected. Below the tabs, there are four input fields: 'USERNAME' with 'adlero', 'DISPLAY NAME' with 'adlero', 'PASSWORD' with six dots, and 'CONFIRM PASSWORD' with six dots. A red 'Create Account' button is located at the bottom.

Figure 20. Create Account

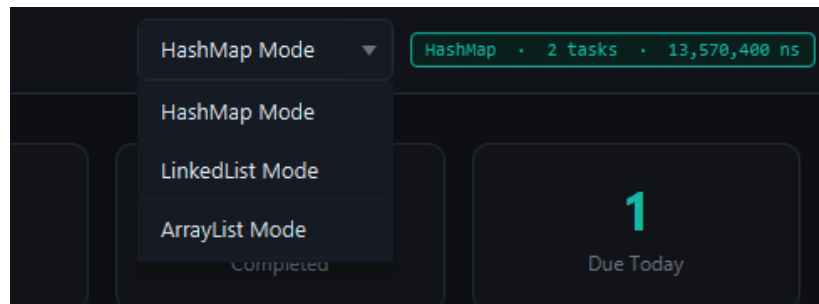


Figure 21. Mode Selection and preview of runtime

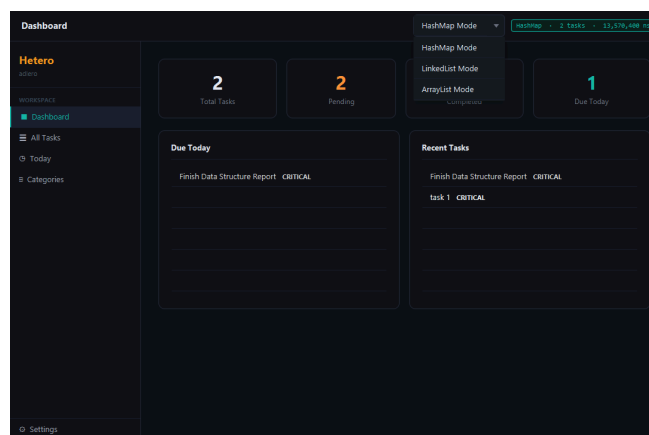


Figure 22. Main dashboard All modes have identical design)

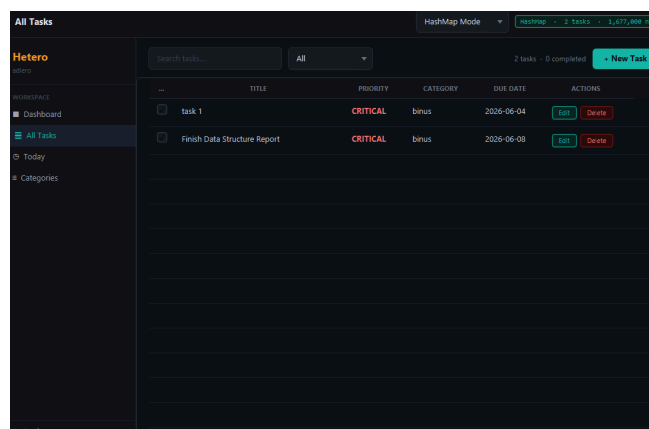


Figure 23. All Task display

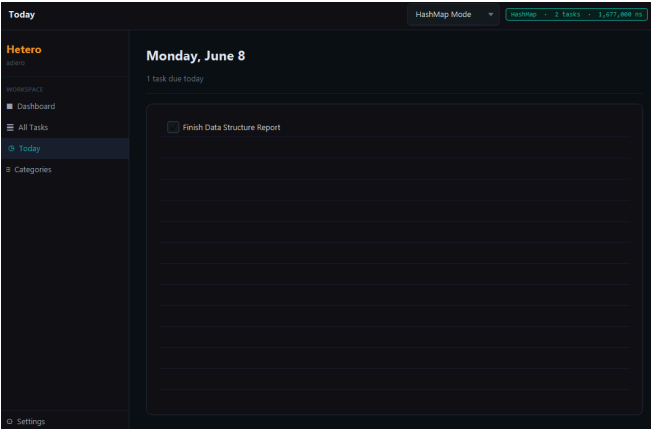


Figure 24. Due today

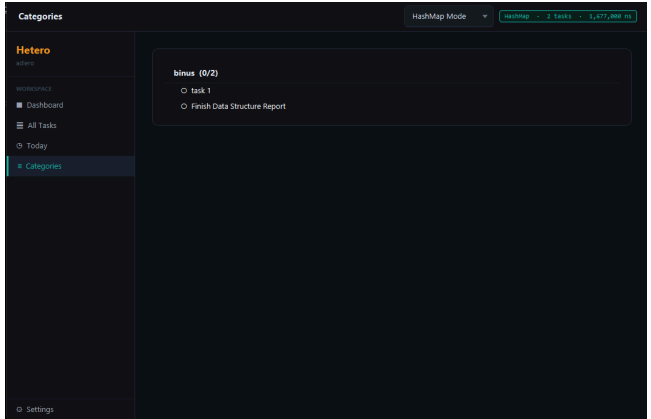


Figure 25. Category selection

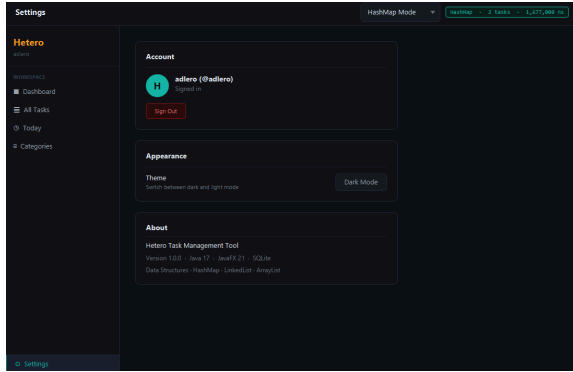


Figure 26. Settings

```
=====
COMPLEXITY ANALYSER & BENCHMARK TOOL
=====
Select a Data Structure to evaluate:
1. ArrayList (ArrayListTaskRepo)
2. LinkedList (LinkedListTaskRepo)
3. HashMap (HashMapTaskRepo)
4. Run All Structures (Side-by-Side Comparison)
5. Exit Program
Enter your choice (1-5):
```

Figure 27. Running a performance test

```
=====
COMPLEXITY ANALYSER & BENCHMARK TOOL
=====
Select a Data Structure to evaluate:
1. ArrayList (ArrayListTaskRepo)
2. LinkedList (LinkedListTaskRepo)
3. HashMap (HashMapTaskRepo)
4. Run All Structures (Side-by-Side Comparison)
5. Exit Program
Enter your choice (1-5): 4
Enter the number of requests/tasks to simulate: 100000
```

Figure 28. Selecting Option 4 and 100,000 operations

```
Running resource usage tracking profile for size: 100000...

===== START COMPARATIVE RUN =====
-----
STRUCTURE PROFILE: ARRAYLISTTASKREPO
-----
Feature Operation Evaluated | Execution (Time) | Heap Space (Delta)
-----
1. Add/Insert Task (Bulk Setup) | 78.3801 ms | 26174.00 KB
2. List All Tasks (findAll) | 2.4387 ms | 901.45 KB
3. Update Task Details (update) | 3.8665 ms | 120.19 KB
4. Find Task by ID (findById) | 3.2178 ms | 120.20 KB
5. Filter by Completion Status | 4.2073 ms | 257.47 KB
6. Filter by Tasks Due Today | 2.9613 ms | 381.02 KB
7. Filter by Category | 2.4589 ms | 844.18 KB
8. Filter by Priority | 1.9418 ms | 566.27 KB
9. Delete Task (delete) | 6.0072 ms | 120.18 KB
-----
STRUCTURE PROFILE: LINKEDLISTTASKREPO
-----
Feature Operation Evaluated | Execution (Time) | Heap Space (Delta)
-----
1. Add/Insert Task (Bulk Setup) | 73.9186 ms | 23451.49 KB
2. List All Tasks (findAll) | 3.3060 ms | 898.14 KB
3. Update Task Details (update) | 4.4040 ms | 116.89 KB
4. Find Task by ID (findById) | 3.0637 ms | 116.89 KB
5. Filter by Completion Status | 3.6859 ms | 254.16 KB
6. Filter by Tasks Due Today | 3.5388 ms | 377.71 KB
7. Filter by Category | 2.8561 ms | 840.88 KB
8. Filter by Priority | 2.5658 ms | 562.97 KB
9. Delete Task (delete) | 5.3917 ms | 116.88 KB
-----
STRUCTURE PROFILE: HASHMAPTASKREPO
-----
Feature Operation Evaluated | Execution (Time) | Heap Space (Delta)
-----
1. Add/Insert Task (Bulk Setup) | 88.3088 ms | 27351.79 KB
2. List All Tasks (findAll) | 2.9261 ms | 877.55 KB
3. Update Task Details (update) | 0.0593 ms | 96.29 KB
4. Find Task by ID (findById) | 0.0227 ms | 96.29 KB
5. Filter by Completion Status | 4.9447 ms | 270.20 KB
6. Filter by Tasks Due Today | 3.4421 ms | 393.74 KB
7. Filter by Category | 4.0252 ms | 856.91 KB
8. Filter by Priority | 3.1043 ms | 579.01 KB
9. Delete Task (delete) | 0.0262 ms | 96.29 KB
-----
===== END COMPARATIVE RUN =====
Press Enter to return to the evaluation menu...

```

Figure 29. Performance Test Results

REFERENCES

- [1]M. Popova, “A Brief History of the To-Do List and the Psychology of Its Success,” The Marginalian, Feb. 09, 2012. Available: <https://www.themarginalian.org/2012/02/09/willpower-to-do-list/>
- [2]V. Bellotti, “What a to-do: studies of task management towards the design of a personal task list manager,” Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 2004. Available: https://www.academia.edu/4323375/What_a_to_do_studies_of_task_management_towards_the_design_of_a_personal_task_list_manager
- [3]J. Pot, “9 best to do list apps of 2021 | Zapier,” zapier.com, Nov. 18, 2022. Available: <https://zapier.com/blog/best-todo-list-apps/>
- [4]Wikipedia Contributors, “Benjamin Franklin,” Wikipedia, Dec. 05, 2018. Available: https://en.wikipedia.org/wiki/Benjamin_Franklin
- [5]D. Allen, Getting Things Done. Penguin, 2015.
- [6]P. Hills, “Productivity loss due to task switching and interruptions.,” International Journal of Information Management, vol. 30, no. 2, pp. 101–102, Apr. 2010, doi: <https://doi.org/10.1016/j.ijinfomgt.2009.12.005>
- [7]N. Bean, “The Chain of Failures: How Data Structures Actually Evolved,” Medium, Jun. 08, 2026. Available: <https://medium.com/@noahbean3396/the-chain-of-failures-how-data-structures-actually-evolved-d16c2afe1c4e>. [Accessed: Jun. 09, 2026]
- [8]GeeksforGeeks, “ArrayList vs LinkedList in Java,” GeeksforGeeks, Oct. 23, 2016. Available: <https://www.geeksforgeeks.org/java/arraylist-vs-linkedlist-java/>
- [9]D. Strmecki, “ArrayList vs. LinkedList vs. HashMap in Java | Baeldung,” www.baeldung.com, Dec. 13, 2020. Available: <https://www.baeldung.com/java-arraylist-vs-linkedlist-vs-hashmap>
- [10]M. Mohamadinia, “Java ArrayList vs LinkedList | Baeldung,” www.baeldung.com, Mar. 27, 2020. Available: <https://www.baeldung.com/java-arraylist-linkedlist>
- [11]“How to Sort a HashMap by Value in Java? | DigitalOcean,” www.digitalocean.com. Available: <https://www.digitalocean.com/community/tutorials/sort-hashmap-by-value-java>
- [12]Oracle, “HashMap (Java Platform SE 8),” Oracle.com, Sep. 11, 2019. Available: <https://docs.oracle.com/javase/8/docs/api/java/util/HashMap.html>
- [13]GeeksforGeeks, “Just In Time Compiler,” GeeksforGeeks, Oct. 14, 2018. Available: <https://www.geeksforgeeks.org/java/just-in-time-compiler/>

[14]GeeksforGeeks, “Internal Working of HashMap in Java,” GeeksforGeeks, May 2017. Available: <https://www.geeksforgeeks.org/java/internal-working-of-hashmap-java/>

[15]baeldung, “Baeldung,” Baeldung on Kotlin, Sep. 10, 2019. Available: <https://www.baeldung.com/java-system-gc>. [Accessed: Jun. 09, 2026]

[16]GeeksforGeeks, “Garbage Collection in Java,” GeeksforGeeks, Feb. 24, 2016. Available: <https://www.geeksforgeeks.org/java/garbage-collection-in-java/>

APPENDIX

1. Program manual

<https://github.com/adlero-clifzier/hetero--tmt/blob/main/HETERO%20Program%20Manual.pdf>

(with screenshots)

There are two versions of the runnable program, PerformanceTest.java (for doing benchmark tests) and HeteroApp.java (The GUI for the visualization of the task manager)

System requirements:

Java Development Kit (JDK): Version 17

Maven: Version 3.9+

Operating System: Compatible with JavaFX 21 (Windows, macOS, Linux with JavaFX support)

Database Engine: SQLite 3.x via JDBC

Libraries

This project requires the following libraries, managed via Apache Maven:

JavaFX 21:

- javafx-controls:21
- Javafx-fxml:21

Testing (JUnit 5):

- junit-jupiter-api:5.10.2
- junit-jupiter-engine:5.10.2

Running PerformanceTest.java (benchmark test)

1. Run PerformanceTest.java.
2. Follow the menu and instructions on the terminal. Tweak the metrics available.

Running HeteroApp.java (GUI app)

1. Run HeteroApp.java
2. Launches HETERO TMT.
3. Dropdown option to switch data structures.
4. Run tests and experiment with different functions. Change the theme via Settings.

2. Github Repository

<https://github.com/adlero-clifzier/hetero--tmt>

3. Poster link

<https://github.com/adlero-clifzier/hetero--tmt/blob/main/HETERO%20-%20Task%20Manager%20App.png>

4. PowerPoint link

<https://github.com/adlero-clifzier/hetero--tmt/blob/main/DS%20and%20OOP%20Presentation.pdf>